263-3300-10L Data Science Lab: Tool-Gym: Tool-Calling Data Generation Pipeline

Suei-Wen Chen*
Department of Computer Science
ETH Zurich, Switzerland

Michal Tešnar*
Department of Computer Science
ETH Zurich, Switzerland

Eduard Ďurech
ETH AI Center, ETH Zurich, Switzerland

Imanol Schlag
ETH AI Center, ETH Zurich, Switzerland

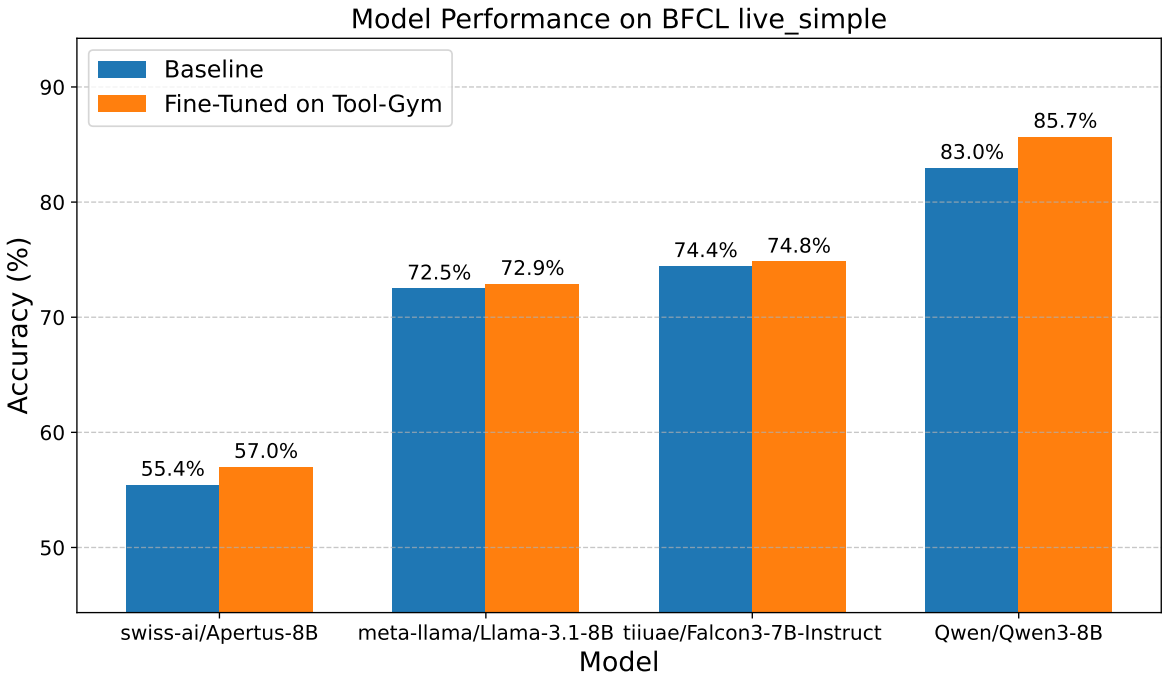


Figure 1: Performance of the models fine-tuned on the Tool-Gym dataset. Using our synthetic tool-calling dataset, we have finetuned the models and evaluated them on the BFCL benchmark [9]

Abstract

We present Tool-Gym, a pipeline for synthetic tool-calling data generation used for LLM post-training. Compared to prior similar efforts, our pipeline generates data based on real-life tools in a more verifiable and scalable manner. Thanks to these properties, our framework can produce high-quality data at scale for supervised fine-tuning as well as reinforcement learning with verifiable rewards. We conduct supervised fine-tuning with low-rank adapters for several models on a training set of 5194 samples. Aside from a test set of 330 samples generated by our pipeline, we also benchmark the fine-tuned models on the Berkeley Function Calling Leaderboard. The efficacy of our framework is demonstrated by the improved accuracy of the fine-tuned models on this external benchmark.

Keywords

Large Language Models, Tool Calling, Function Calling, Supervised Fine-Tuning, Reinforcement Learning

*Equal contribution.

Preprint. This course project work can be distributed as a preprint and has not been peer-reviewed. It does not constitute archival publication and remains eligible for submission to academic venues, including workshops, conferences, and journals.
License. The authors grant ETH Zurich and the ETH AI Center a non-exclusive license to display this work on their platforms to showcase student projects. Redistribution or publication by others outside of academic venues requires the consent of the authors.
Code. Associated code is available open-source and under the MIT License, which permits free reuse, free modification, and free distribution, provided proper attribution is given to the authors, and no liability is assumed by the authors. Follow up work is not required to be open-source and is not required to have an MIT License. The code and its MIT license are publicly available here:
https://github.com/swiss-ai/tool-gym

263-3300-10L Data Science Lab, October 10, 2026, Zurich, Switzerland
© 2025 Copyright held by the owner/author(s).

Author contributions using the CRediT framework [1]:
SC: Methodology, Software, Visualization, Investigation, Writing - Original Draft
MT: Methodology, Software, Visualization, Investigation, Writing - Original Draft

1 Introduction

Large Language Models (LLMs) are increasingly used to perform a wide range of tasks with greater autonomy and less human supervision. This is usually done by providing LLMs with access to external tools (or functions) to utilize, and the quality of agentic workflow critically relies on the model's ability to call tools correctly. However, even frontier models may fail to be reliable, even on tasks involving only one function call [9]. One strategy to increase models' tool calling ability is to conduct post-training on high-quality dataset. Specifically, models should first go through supervise fine-tuning (STF) for the correct formatting to call tools, and then trained with reinforcement learning (RL) for more targeted and accurate tool usage.

With this motivation, we set out to create Tool-Gym, a framework that enables generation of data for both SFT and RL with the help of strong LLMs. On a high level, the dataset consists of a task which can be solved with some given tools, the ground truth answer for tool usage and final output, and intermediary components which make the verification possible. We set ourselves the ambitious goal of creating a dataset that is (1) close to real world data and diverse—so that models can perform well in day-to-day use cases, (2) verifiable in its correctness—so that we can safely train on this data and expect improvements, and (3) able to be mass generated with minimal human labor—so that models can learn from more examples.

However, a brief thought experiment reveals that the coexistence of these three desiderata is a near impossibility if done naively via LLM prompting alone. To achieve diversity and verifiability, one has to implement verification for the outputs of numerous real tools and their compositions, which makes large-scale data generation challenging without significant human effort. On the other hand, if verification is implemented for only a few tools from which a lot of data is generated, the diversity will be small and we run the risk of overfitting our models. Lastly, if we take a large number of tools and generate a diverse dataset from them, we will not be able to verify the data and ensure its quality due to tool argument constraints.

Tool-Gym addresses all three requirements without comprise: it is a pipeline which allows us to mass-generate realistic data while being verifiable. To ensure (1), we source the tools from open source repositories on GitHub, providing a diverse set of set of real-life tools. To ensure (2), we inspect each tool manually to constrain the possible inputs and implement samplers for them. Finally, to ensure (3), the tools are randomly sampled and combined to create diverse question templates which can be parametrized using argument samplers, thus generating virtually infinite amount of data.

2 Related Works

2.1 Model Context Protocal

Model Context Protocol (MCP) is an widely adopted open-source standard for connecting AI applications to external systems [5]. MCP defines a standard interface for LLM clients to utilize tools, solving the integration problem between application needs and tool definitions.

ED: Conceptualization, Supervision
IS: Project Administration

2.2 Synthetic Data Generation

Toucan [11] is the largest existing SFT dataset for tool calling, with 1.5 million examples constructed from a wide range of open-source MCP servers on GitHub. However, their pipeline only achieves very weak verifiability, as only LLM-as-judges is used to assess the quality of data samples, and their ground truth contains only the target tools without input arguments.

APIGen [7] tries to ensure stronger verifiability and diversity than Toucan at the expense of a smaller dataset of 60,000 samples. Their approach is similar to ours in spirit, but the design of the templated prompt in Tool-Gym enables complex composition of tools which APIGen does not support.

ToolACE [6] introduces nested tool calls along with other improvements upon APIGen, but their generation pipeline relies on a complex iterative agentic workflow. This means the throughput of ToolACE is constrained by LLM inference in both time and cost, whereas Tool-Gym relies on argument samplers which can achieve mass-generation with little computational overhead, making Tool-Gym a superior pipeline for RL data generation.

2.3 Benchmarks

There are several popular benchmarks to assess the tool calling capabilities of LLMs. MCP-Universe [8] is the first such benchmark, spanning only 11 MCP servers. τ-Bench [13] focuses on agent provided with domain-specific API tools and policy guidelines. ACEBench [2] assesses situations with ambiguous or incomplete instructions as well as multi-turn dialogues. BFCL [9] evaluates diverse scenarios including parallel, multi-step, multi-turn and agentic tool use, with possibly no correct tool offered to the model for a given task.

3 Data Generation

This section describes in detail the Tool-Gym data generation pipeline, which is the main contribution of this work. We explain both the technical and the conceptual aspects. We start by describing the source of our tools, how they were filtered and which servers and tools we ended up selecting. Afterwards, we describe how these tools are used in combination with strong LLMs to generate data for both supervised fine-tuning and reinforcement learning of LLMs. Finally, we give the specifics of the dataset we generated for further experiments and analysis.

3.1 MCP Image Build Pipeline

To generate data with the highest fidelity, we took open-source MCP servers from GitHub. To efficiently locate high-quality candidate servers, we resort to those enlisted by Toucan [11]. Specifically, the list starts from a large number of scraped servers, which are then filtered down to only those passing some checks. This leaves us with a list of 570 servers with corresponding links to Smithery[1], a platform for developers to find and ship language model extensions compatible with MCP.

Due to considerations such as reproducibility and being able to run MCP servers on demand without relying on a third party like Smithery, it is desirable to include the servers as part of the pipeline which can be invoked locally. This is done by downloading

[1] https://smithery.ai/

all the servers and run them using a simple abstraction. Specifically, we have built an automated stack to (1) clone the corresponding repositories from GitHub, (2) build the corresponding docker image using Podman[2] if available, (3) test running the docker image, and (4) automatically discovers the available MCP tools using the standard input of the running image.

We came across several issues when creating this automation stack. Firstly, the open source tools usually contain configuration file for a custom build system from Smithery, however, this system is not open source which makes it hard to rebuild the images. Secondly, some of the servers use NPM[3] to build images. In addition, some of the images fail to build. As there is not a fixed time when tools have been used in Toucan, some have been updated or discontinued, or they do not fulfill build requirements. Even if an image builds, it may fail to run. Moreover, some of the servers use HTTP instead of stdio for communication, thus causing failure. Finally, some of the servers do not offer any discoverable tools. This drastically reduced the number of usable servers to 153, but we have a fully automated pipeline for building images which serve our purpose. The amount of implemented servers is summarized in Figure 2.

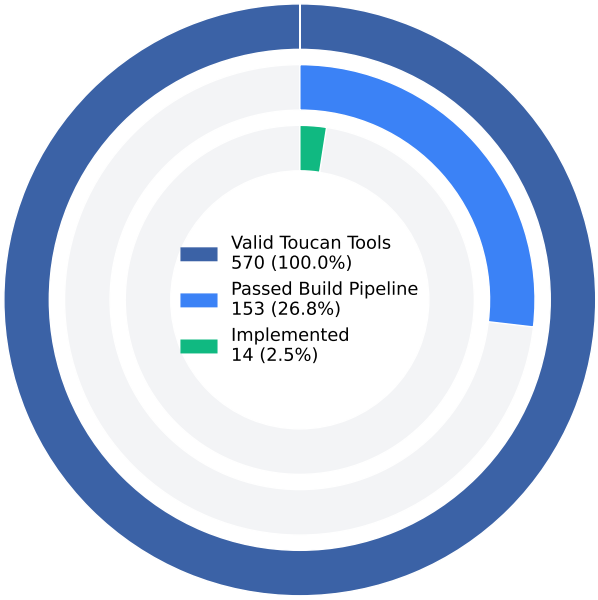


Figure 2: Servers Implemented in Tool-Gym. Out of 570 servers that were scraped, only 153 were build for our purposes. Finally, we have managed to fully implemented 14, giving us access to 47 tools in total for generation.

Among the 153 servers, we carefully select the ones to include in our Tool-Gym generation pipeline by aiming for maximum diversity. Certain functionalities such as weather tools or calculators have many different implementations, so we make sure to only include one server from each category. Furthermore, we had to filter out tools that were not safe for fine-tuning. These include some servers which engage in illegal or immoral practices to scrape websites, as well as some servers which provide dummy implementation only and do not contain the proper execution logic. Overall, we have implemented 14 servers with 47 tools. Table 1 summarizes the implemented servers.

Table 1: MCP Servers Implemented in Tool-Gym. From tools that passed the build checks, we have selected tools that cover different domains to create diversity in the data. The 14 implemented servers together hold 47 unique tools.

Server	Description	# Tools
`alan5543/calculator-mcp`	Basic math operations	17
`daheepk/arxiv-paper-mcp`	Arxiv search and analysis	3
`hcatakli/dictionary-mcp`	Word definition retrieval	1
`HarunGuclu/bible-mcp`	Verses, chapters	4
`ceydasimsekk/book_mcp`	Book title search	1
`yokingma/time-mcp`	Time zones and formats	6
`myrve/MCP-Fruit`	Bio and nutritional info	1
`sellisd/mcp-units`	Unit conversion	3
`BioContext/PubChem-MCP`	Compound properties.	1
`ForceConstant/lyrical_mcp`	Get rhymes, count syllables	2
`esmakymkci/yeni_Detect_lang_mcp`	Language detection	1
`MetehanGZL/PokeMCP`	Generates random Pokemon	3
`mikechao/metmuseum-mcp`	Met Museum information	3
`IlaydaBayrak/simple-mcp`	Reverses text	1
Total	–	47

For ease of use, set of commands on servers and their images has been abstracted. Figure 3 gives an overview of the sequence of steps involved. The command `save` saves a built image into local storage, whereas `load` loads it from there to `podman images`. The command

[2]https://podman.io/
[3]https://docs.npmjs.com/

`build` includes everything from cloning, building and image and storing it into local storage. Each respective step is skipped if given operation does not need to be done. The solid lines in Figure 3 signify the actual intended way of using the command. The dotted lines signify that the `build` command will do these or skip these if they are not necessary.

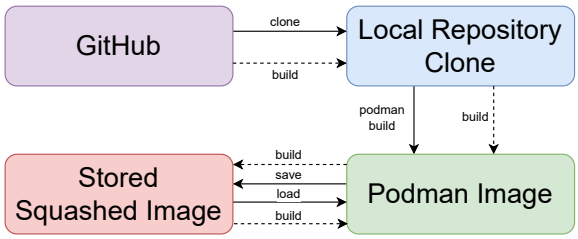


Figure 3: Photography Commands. The utilities implemented for managing and installing servers operate on 4 different stages: repositories are cloned, then docker images are build and then stored for later loading.

3.2 Data Synthetization Process

Figure 4 shows the Tool-Gym data generation workflow, which relies on a strong pretrained model as the backbone for generation and covers both SFT and RL.

The main idea behind the workflow is to combine task templates with argument samplers to generate practically infinite data. The first step is to randomly sample one or more tools and prompt a strong model to generate a "templated task" which involves a question template, an argument schema and a solution script. We describe each of them below:

- The question template is a question in natural language which can and should be solved using the provided tool(s), except that the tool arguments are variable names instead of their actual values.
- The solution script is the intended solution to the question template in python code: given the values for the variables in the question template, upon execution the code calls tools with the correct arguments and returns the final result.
- The mapping between the variable name in the question template and the input argument names is given by the argument schema. For composability, we also record the output type of a tool if its output is fed as an input downstream. This is needed for the solution script to cast tool outputs to its intended type.

We use few-shot examples to guide the strong model to follow specific output formats for the above, with XML tags enclosing each part of the output. The prompt, the examples and the detailed format for argument schema can be found in Appendix A.

After obtaining the argument schema, we then call our custom type samplers to get a set of tool input values and plug them into both the question template and the solution script. Once we do this, we obtain a question which can readily be solved by an LLM, along with its solution script which gives the ground truth answer. From

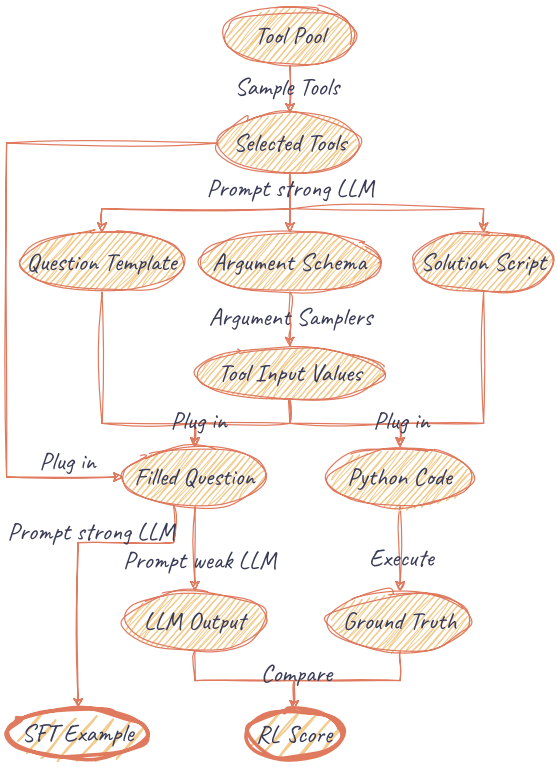


Figure 4: Data Generation Workflow for a Single Task. The tools for a particular task are sampled from the pool of implemented tools. Subsequently, a large pretrained LLM is prompted to generated a task for this combination of tools, an argument schema and correct solution script using a few-shot examples. The template, and the solution script are filled in using valid sampled values. The solution is executed to create a ground truth answer, while the filled question can be passed to a LLM which performs RL training to later compare answer. Alternatively, one can once again use a large pretrained LLM to answer the task, which can then be used as a sample for SFT.

here, we can do both SFT and RL: SFT data can simply be generated from prompting the strong model to answer the question, whereas RL can be done by taking whether the output of the weak LLM matches the execution output of the python code as the reward.

Unlike all previous works in which LLMs come up with tool input arguments, the novelty of Tool-Gym lies in replacing the tool arguments with placeholders with type constraints when the strong model generates a question, and to this end, we must delegate the responsibility of handling input validity to the developers. We inspect each tool and determine the appropriate (custom) type of each input argument and its output. This may involve getting predefined values from an API or build a class implementing the type constraints. Once a custom type is defined, a sampler needs to be implemented on top of it. As an example, to sample from

the type `ArxivID` representing a valid arXiv identifier, one would need to sample a year and a month, make an API call to get the last submission of the month to know how many paper was submitted in total, sample a number less than that, retrieve the corresponding paper and sample a particular version if there are more than one.

Once custom types and samplers have been implemented for a tool, we substitue the new data types into the JSON schema of the tool following OpenAI standards. The updated schema is then fed into the strong LLM to generation tasks. Notice that when models are asked to answer a generated question, tool information is provided in the original tool schema to faithfully reflect real-world usage.

Conceptually, our approach of templating tasks with type constraints has the following advantages:

(1) Tools in real life are diverse, and LLMs may not have good judgments on the valid input values for each tool. This can be the case for highly specialized tools which the model has not been exposed to during training. However, this may even happen for common tools expecting different formats representing the same information, e.g. location or time data for weather retrieval. Relying on LLMs to come up with the correct input formats and values by guessing destroys verifiability, since tool calls will fail to execute in the background.
(2) The filled question may not follow a fairly improbable decoding path of the strong LLM given that the space of valid argument combination can be vast. With exposure to unlikely continuations during SFT, the model may be able to reduce hallucination caused by outputting the most probable but wrong continuation when decoding input arguments.
(3) One generated template may give rise to a large amount of data by varying the input argument. The human effort required to implement type constraints and samplers scales linearly with the number of tools, but the possible realizations of filled tasks grow combinatorially with the number of valid input combinations. This realizes the goal of maximizing effect with minimal manual labor.

3.3 Generated Datasets

For simplicity, we have decided as the first step to focus on the training and evaluation of the simplest form, namely data with just one tool call per question. This means that the number of tool sampled in the pipeline is set to 1. We do this so that we can fine-tune for and make a fair comparison with the simplest task on the Berkeley Function-Calling Leaderboard called `live_simple`, which contains open source contributions of different tasks concerning one single tool call.

We use Qwen/Qwen3-Next-80B-A3B-Thinking [12] as our strong pretrained LLM. For low-rank fine-tuning, we generate 5194 training samples. For evaluation, we generate 330 testing tasks using our pipeline and structure them into the same format as in BFCL to be able to use their backend for evaluation.

It was decided that the tool calling data should be generated according to the format of Apertus[4] to faciliate future post-traning effort. This introduced some complications as most LLMs provided by HuggingFace use a different format[5] and therefore the dataset had to be converted. Subsequently, before tokenization, each other models applies its respective chat template to the data, which results in the final strings passed to the models in their respective format to continue generation.

4 Supervised Fine-Tuning

With the aim of finetuning `swiss-ai/Apertus-8B-Instruct-2509`[6] [4], we selected for fair comparison 3 models of the same size, which are all instruct models.

- `meta-llama/Llama-3.1-8B-Instruct`[7] [3]
- `tiiuae/Falcon3-7B-Instruct`[8] [10]
- `Qwen/Qwen3-8B`[9][12]

We train using learning rate 10^{-4}, logical batch size 32 (4 per device, 8 gradient accumulation steps). The LoRA setting were $r = 16$, $\alpha = 32$, dropout = 0.1, targeting all modules. The scheduler was cosine with minimum learning rate 0.1 and warm up ratio 0.05. The models are trained for 1 and 2 epochs, both checkpoints are taken and evaluated. The performance of the better one is presented.

5 Results

We conduct our evaluation on the `live_simple` subset of the Berkeley Function-Calling Leaderboard [9], which contains 258 diverse open-source contributed tasks requiring one correct tool call to be solved. The score is computed as the percentage of correct final output. The performance is visualized in Figure 1.

To evaluate the performance of the models in distribution of the training data, we have formatted a held-out test set of the Tool-Gym data to the format accepted by BFCL, which we refer to as the Tool-Gym benchmark. By doing so, we leverage the implementation of tool call extraction from the LLM output and scoring mechanisms of BFCL to our generated data.

We report the results of both BFCL and the Tool-Gym benchmark in Table 2. All the models selected for fine-tuning see performance gain on the BFCL benchmark. In particular, `Qwen3-8B` exhibits the biggest performance gain of 2.71% despite already being the most performant model in the pool before fine-tuning, followed by `Apertus-8B-Instruct-2509` with 1.55%. As for the in-distribution Tool-Gym benchmark, all the models achieve significantly higher accuracy than in BFCL, with `Apertus-8B-Instruct-2509` and `Falcon3-7B-Instruct-FC` getting some performance gain while the other two models stay the same.

6 Discussion

In this work we introduce Tool-Gym, a pipeline which generates tool calling data for practical use cases in a scalable and verifiable way. One of the biggest advantages of Tool-Gym is that with the architecture we implemented, a developer may take any open-source MCP server online and turn it into fine-tuning data by implementing custom type details and sampling. We evaluate the efficacy of

[4] https://github.com/swiss-ai/apertus-format/tree/main

[5] https://huggingface.co/blog/unified-tool-use
[6] https://huggingface.co/swiss-ai/Apertus-8B-Instruct-2509
[7] https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct
[8] https://huggingface.co/tiiuae/Falcon3-7B-Instruct
[9] https://huggingface.co/Qwen/Qwen3-8B

Table 2: Model Performance on BFCL live_simple and Tool-Gym Datasets

Model	live_simple	Tool-Gym
swiss-ai/Apertus-8B-Instruct-2509	55.43%	90.30%
swiss-ai/Apertus-8B-Instruct-2509-SFT	56.98%	91.21%
meta-llama/Llama-3.1-8B-Instruct	72.48%	95.76%
meta-llama/Llama-3.1-8B-Instruct-SFT	72.87%	95.76%
tiiuae/Falcon3-7B-Instruct-FC	74.42%	93.64%
tiiuae/Falcon3-7B-Instruct-FC-SFT	74.81%	95.15%
Qwen/Qwen3-8B	82.95%	96.36%
Qwen/Qwen3-8B-SFT	85.66%	96.36%
meta-llama/Llama-3.1-8B	7.36%	0.00%
meta-llama/Llama-3.1-8B-SFT	25.58%	36.36%

our framework using both the external BFCL benchmark and our in-distrubion Tool-Gym benchmark. The selected baseline models all show improvement on BFCL after fine-tuning, and they already achieve high accuracy on Tool-Gym benchmark before fine-tuning. We suspect that this is because our generated data is relatively easy and less diverse compared to the `live_simple` segment of BFCL. We also remark that the selected base models have all been trained for tool-calling, so we do not expect to see significant improvement after fine-tuning on a small dataset. We believe that by adding more MCP servers into the data generation pipeline and increase the size of our synthetic data, our proposed framework can achieve significant improvement on external benchmarks.

For comparison, we add the results in Figure 2 of fine-tuning on one base model, namely `meta-llama/Llama-3.1-8B`[10]. We note that the performance of that model increases significantly. This could be due to the fact that the model is not used to the conversation format, however, it gives us strong hopes that the data we have generated aligns well with the goals of creating tool calling dataset.

7 Future Work

We point out a number of directions as the logical next steps to extend this work, improving the quality of the data generation process as well as the evaluations.

7.1 Data Generation Pipeline

As noted, the amount of MCP servers that could be implemented is limited by the choice of supporting only servers provided as docker images with stdio communication. In next steps, we would like to extend the support for different types of images, especially those using HTTP to and those that are build using NPM. Moreover, implementing more custom types and samplers for more servers and tools overall will increase the quantity and diversity of the generated data. However, this is quite tedious and therefore for the first version of the pipeline was not a priority, but remains a crucial extension for the future.

To teach the model when to call tools and when not to call tools, we would like to add negative examples (tasks which do not require tools or for which no correct tool is provided) into the data generation process. We believe this will help combat overfitting and improve the overall usability of the final model.

Finally, both in generations and evaluations, we would like to add support for multi-turn and parallel tool calling to assess the efficacy of our pipeline for more realistic use cases. In particular, adding multi-turn would require simulating back-and-forth conversation with the user using a strong model.

7.2 Training and Evaluation

During the development of the evaluation of the pipeline, we have realized that evaluation the quality of a dataset is not straight forward. It certainly stays open to properly evaluate the benefit of complex tool composition on multi-step questions enabled by our data generation pipeline. This would also require a more extensive evaluation on the entire BFCL benchmark and in detail analysis of the results of produced by the model.

Even more importantly, what remains to be done is performing training and evaluation using the proposed RL pipeline, which is the motivation inspiring the design of Tool-Gym in the first place.

Eventually, the pipeline should also be evaluated using bigger models such as `swiss-ai/Apertus-70B-Instruct-2509`[11]. Due to time constraints, this work only focuses on smaller models which have more limited capabilities. For such purposes also a bigger amount of data should be generated.

Acknowledgments

Great thanks to Dana Shayakhmetova, Loic Deslarzes and Morgane Magnin, EPFL Master's students who were involved in the implementation of the project.

We would like to also express gratitude to ETH AI Center awesome cappuccinos during the duration of the project.

This work was supported as part of the *Swiss AI initiative* by a grant from the Swiss National Supercomputing Centre (CSCS) on Alps.

[10] https://huggingface.co/meta-llama/Llama-3.1-8B

[11] https://huggingface.co/swiss-ai/Apertus-70B-Instruct-2509

References

[1] Amy Brand, Liz Allen, Micah Altman, Marjorie Hlava, and Jo Scott. Beyond authorship: Attribution, contribution, collaboration, and credit. *Learned Publishing*, 28(2):2, 2015.

[2] Chen Chen, Xinlong Hao, Weiwen Liu, Xu Huang, Xingshan Zeng, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Yuefeng Huang, et al. Acebench: Who wins the match point in tool usage? *arXiv preprint arXiv:2501.12851*, 2025.

[3] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[4] Alejandro Hernández-Cano, Alexander Hägele, Allen Hao Huang, Angelika Romanou, Antoni-Joan Solergibert, Barna Pasztor, Bettina Messmer, Dhia Garbaya, Eduard Frank Ďurech, Ido Hakimi, et al. Apertus: Democratizing open and compliant llms for global language environments. *arXiv preprint arXiv:2509.14233*, 2025.

[5] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*, 2025.

[6] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, et al. Toolace: Winning the points of llm function calling. *arXiv preprint arXiv:2409.00920*, 2024.

[7] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh RN, et al. Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37:54463–54482, 2024.

[8] Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. Mcp-universe: Benchmarking large language models with real-world model context protocol servers. *arXiv preprint arXiv:2508.14704*, 2025.

[9] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.

[10] Falcon-LLM Team. The falcon 3 family of open models, December 2024. URL https://huggingface.co/blog/falcon3.

[11] Zhangchen Xu, Adriana Meza Soria, Shawn Tan, Anurag Roy, Ashish Sunil Agrawal, Radha Poovendran, and Rameswar Panda. Toucan: Synthesizing 1.5 m tool-agentic data from real-world mcp environments. *arXiv preprint arXiv:2510.01179*, 2025.

[12] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[13] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Appendices

A Prompts

Here is the templated prompt we used on the strong pre-trained model to generate templated tasks and argument schema:

```
You are an expert LLM trainer which comes up with tasks to improve the tool calling capablity of LLMs.
Below you will be given a set of tools with descriptions of their functions and parameters. Your job is to come up with an
appropriate task for an LLM to solve by calling these tools. Make sure that the task you generate can be answered only by
the tools you are provided with and that the parameter types are respected.
IMPORTANT: Do not include steps which require calling tools not provided to you or additional calculations. Sometimes this
means you should only use some of the provided tools, but try to utilize as many tools as possible.
IMPORTANT: Do not call tools or do any computation yourself, since this is the job of the LLM being trained.
IMPORTANT: When generating the input data, ALWAYS copy the data type VERBATIM from the tool description.
IMPORTANT: In the solution_script, ALWAYS call `extract_typed_result()` after `execute_mcp_tool()`
IMPORTANT: The final answer must be ONE output of the last tool call.
The format of your output must adhere to the following structure:
<answer>
  <task>
    <!--Provide an appropriate task here.-->
  </task>
  <input_data>
    <!--Provide the input data required to solve the task as a valid JSON object as specified by the parameters
    field in the tool descriptions.
    Importantly, the top level elements must be {tool_name: params}, and params must be a dictionary with keys being
    the names for the tool arguments and values being the placeholder variable names in the task.
    -->
  </input_data>
  <tool_output_type>
    <!--Provide the JSON mapping from all the provided tool names to their output types directly from the tool descriptions-->
  </tool_output_type>
  <solution_script>
    ```python
    async def task(input_data, tool_output_type, server_name):
      # Provide the intended solution for the task in python here
      # input_data comes from the JSON you generated in the <input_data/> element
    ```
  </solution_script>
</answer>

Here are a few examples:
{{examples}}

Now start!
Available tools
{{tools}}
Output
```

Here is the snippet containing the few-shot examples which we fed into the prompt above:

```
       # Example 1
## Available tools
[
  {
    "type": "function",
    "function": {
      "name": "is_prime",
      "description": "Check if a number is prime",
      "parameters": {
        "properties": {
          "n": {
            "title": "N",
            "type": "integer"
          }
        },
        "required": [
          "n"
        ],
        "title": "is_primeArguments",
        "type": "object"
      },
      "output": {
        "type": "boolean"
      }
    }
  },
  {
    "type": "function",
    "function": {
      "name": "gcd",
      "description": "Calculate greatest common divisor of two integers",
      "parameters": {
        "properties": {
          "a": {
            "title": "A",
            "type": "integer"
          },
          "b": {
            "title": "B",
            "type": "integer"
          }
        },
        "required": [
          "a",
          "b"
        ],
        "title": "gcdArguments",
        "type": "object"
      },
      "output": {
        "type": "integer"
      }
    }
  }
]
```

```
## Output
<answer>
  <task>
    Is the greatest common divisor of <var type="integer">var_a</var> and <var type="integer">var_b</var> a prime number?
  </task>
  <input_data>
    {
      "gcd": {
        "a": "var_a",
        "b": "var_b"
      }
    }
  </input_data>
  <tool_output_type>
    {
      "gcd": "integer",
      "is_prime": "boolean"
    }
  </tool_output_type>
  <solution_script>
    ```python
    async def task(input_data, tool_output_type, server_name):
      gcd_result = await execute_mcp_tool(server_name, tool_name="gcd", tool_args=intput_data["gcd"])
      gcd_result = extract_typed_result(gcd_result, tool_output_type["gcd"])
      is_prime_result = await execute_mcp_tool(server_name, tool_name="is_prime", tool_args={"n": gcd_result})
      is_prime_result = extract_typed_result(is_prime_result, tool_output_type["is_prime"])
      return is_prime_result
    ```
  </solution_script>
</answer>

Example 2
Available tools
[
  {
    "type": "function",
    "function": {
      "name": "cos",
      "description": "Calculate cosine of an angle in radians",
      "parameters": {
        "properties": {
          "x": {
            "title": "X",
            "type": "number"
          }
        },
        "required": [
          "x"
        ],
        "title": "cosArguments",
        "type": "object"
      },
      "output": {
        "type": "number"
      }
    }
```

```
  },
  {
    "type": "function",
    "function": {
      "name": "tan",
      "description": "Calculate tangent of an angle in radians",
      "parameters": {
        "properties": {
          "x": {
            "title": "X",
            "type": "number"
          }
        },
        "required": [
          "x"
        ],
        "title": "tanArguments",
        "type": "object"
      },
      "output": {
        "type": "number"
      }
    }
  },
  {
    "type": "function",
    "function": {
      "name": "quadratic_roots",
      "description": "\n Solve quadratic equation ax\u00b2 + bx + c = 0\n Returns a tuple of roots (real or complex)\n ",
      "parameters": {
        "properties": {
          "a": {
            "title": "A",
            "type": "number"
          },
          "b": {
            "title": "B",
            "type": "number"
          },
          "c": {
            "title": "C",
            "type": "number"
          }
        },
        "required": [
          "a",
          "b",
          "c"
        ],
        "title": "quadratic_rootsArguments",
        "type": "object"
      },
      "output": {
        "type": "array"
      }
    }
  }
```

````
]
## Output
<answer>
  <task>
    Solve the quadratic equation $\cos(t)x^2 + t x + \tan(t) = 0$, where $t=<var type="float">t_val</var>$.
  </task>
  <input_data>
    {
      "cos": {
        "x": "t_val"
      },
      "tan": {
        "x": "t_val"
      },
    }
  </input_data>
  <tool_output_type>
    {
      "cos": "number",
      "tan": "number",
      "quadratic_roots": "array"
    }
  </tool_output_type>
  <solution_script>
    ```python
    async def task(input_data, tool_output_type, server_name):
      cos_result = await execute_mcp_tool(server_name, tool_name="cos", tool_args=intput_data["cos"])
      cos_result = extract_typed_result(cos_result, tool_output_type["cos"])
      tan_result = await execute_mcp_tool(server_name, tool_name="tan", tool_args=intput_data["tan"])
      tan_result = extract_typed_result(tan_result, tool_output_type["tan"])
      quadratic_roots_result = await execute_mcp_tool(server_name, tool_name=quadratic_roots,
            tool_args={"a": cos_result, "b": t_val, "c": tan_result})
      quadratic_roots_result = extract_typed_result(quadratic_roots_result, tool_output_type["quadratic_roots"])
      return quadratic_roots_result
    ```
  </solution_script>
</answer>
````

B Example of Generated Task

For illustration, an example of generated task by the strong LLM is presented below. This contains the generated task and their execution.

```
************************** GENERATION DEMO **************************
Generating a new task template using the teacher LLM for server alan5543_calculator_mcp...
USING RANDOM TOOLS: ['lcm', 'is_prime']

Generated Task:
 <answer>
  <task>
    Is the least common multiple of <var type="integer">var_x</var> and <var type="integer">var_y</var> a prime number?
  </task>
  <input_data>
    {
      "lcm": {
        "a": "var_x",
        "b": "var_y"
      }
    }
  </input_data>
  <tool_output_type>
    {
      "lcm": "integer",
      "is_prime": "boolean"
    }
  </tool_output_type>
  <solution_script>
    ```python
    async def task(input_data, tool_output_type, server_name):
      lcm_result = await execute_mcp_tool(server_name, tool_name="lcm", tool_args=input_data["lcm"])
      lcm_result = extract_typed_result(lcm_result, tool_output_type["lcm"])
      is_prime_result = await execute_mcp_tool(server_name, tool_name="is_prime", tool_args={"n": lcm_result})
      is_prime_result = extract_typed_result(is_prime_result, tool_output_type["is_prime"])
      return is_prime_result
    ```
  </solution_script>
</answer>

Saved generated question to generated_questions/alan5543_calculator_mcp-ab0127f1-e048-48a3-99ca-310b4477bbe4.json

************************ SCRIPT EXECUTION DEMO ************************

Using server: alan5543_calculator_mcp, output UUID: ab0127f1-e048-48a3-99ca-310b4477bbe4
Sampled Tools: ['lcm', 'is_prime']
Teacher Generated Output:
Task: Is the least common multiple of <var type="integer">var_x</var> and <var type="integer">var_y</var> a prime number?
Input data scheme: {'lcm': {'a': 'var_x', 'b': 'var_y'}}
Tool output types: {'lcm': 'integer', 'is_prime': 'boolean'}
Solution Script:
async def task(input_data, tool_output_type, server_name):
      lcm_result = await execute_mcp_tool(server_name, tool_name="lcm", tool_args=input_data["lcm"])
      lcm_result = extract_typed_result(lcm_result, tool_output_type["lcm"])
      is_prime_result = await execute_mcp_tool(server_name, tool_name="is_prime", tool_args={"n": lcm_result})
      is_prime_result = extract_typed_result(is_prime_result, tool_output_type["is_prime"])
      return is_prime_result

=========== Executing the generated task with random data ===========
```

```
Executing tool 'lcm' via Docker MCP client for arguments: {'a': 2, 'b': 9}
MCP tool executed successfully. Result: meta=None content=[TextContent(type='text', text='18', annotations=None, meta=None)]
    structuredContent={'result': 18} isError=False
Executing tool 'is_prime' via Docker MCP client for arguments: {'n': 18}
MCP tool executed successfully. Result: meta=None content=[TextContent(type='text', text='false', annotations=None, meta=None)
     ]
    structuredContent={'result': False} isError=False

========================= Execution Result =========================

Execution Succeeded. Result: {'ground_truth': False, 'filled_task': 'Is the least common multiple of 2 and 9 a prime number?'}

************************ STUDENT EXECUTION DEMO ************************

==================================================
Automatically executing pre-generated task.
==================================================
Executing tool 'lcm' via Docker MCP client for arguments: {'a': 10, 'b': 7}
MCP tool executed successfully. Result: meta=None content=[TextContent(type='text', text='70', annotations=None, meta=None)]
    structuredContent={'result': 70} isError=False
Executing tool 'is_prime' via Docker MCP client for arguments: {'n': 70}
MCP tool executed successfully. Result: meta=None content=[TextContent(type='text', text='false', annotations=None, meta=None)
     ]
    structuredContent={'result': False} isError=False
==================================================
Letting student LLM execute the filled task.
==================================================
Student was given prompt:
Is the least common multiple of 10 and 7 a prime number?
Obtain your final answer by copying the content inside the TextContent.text field of your final tool call output. Wrap
    your final answer into <answer>YOUR_ANSWER_HERE</answer>.

Student wants to call 1 tool(s):
  > Executing lcm with args: {'a': 10, 'b': 7}
  > Result: meta=None content=[TextContent(type='text', text='70', annotations=None, meta=None)]
       structuredContent={'result': 70} isError=False

Student wants to call 1 tool(s):
  > Executing is_prime with args: {'n': 70}
  > Result: meta=None content=[TextContent(type='text', text='false', annotations=None, meta=None)]
       structuredContent={'result': False} isError=False

Final Student Response: <answer>false</answer>
Student Execution Result:
Ground Truth: False
Ground Truth Tools Used: ['lcm', 'is_prime']
Student Response: False
Student Tools Called: ['lcm', 'is_prime']
Whether Student Output Matches Ground Truth: True
Whether Student Used Correct Tools: True
```